

DDD workshop

PXL 2026

Programma

Dag 1 — De bouwstenen in code

1. DDD: tackling complexity · tactical vs strategic (terugblik strategic-sessie)
 2. Value objects & entities
 3. Aggregates: commands, rules en events + testen
 4. Factories, repositories & domain services
 5. Onion architecture & application services
-

Dag 2 — Van aggregate tot werkende applicatie

1. Herhaling dag 1
2. Projectstructuur, application layer en api/console
3. Persistentie: state stored vs event sourcing
4. Messaging: commands, notifications, sync vs async
5. Projections & CQRS



Spreker Daniël
Mertens

Over de spreker

Daniël Mertens — consultant bij Info Support, 7 jaar actief in DDD. Padel, coding puzzles.

Contact: daniel.mertens@infosupport.com

Enkele regels

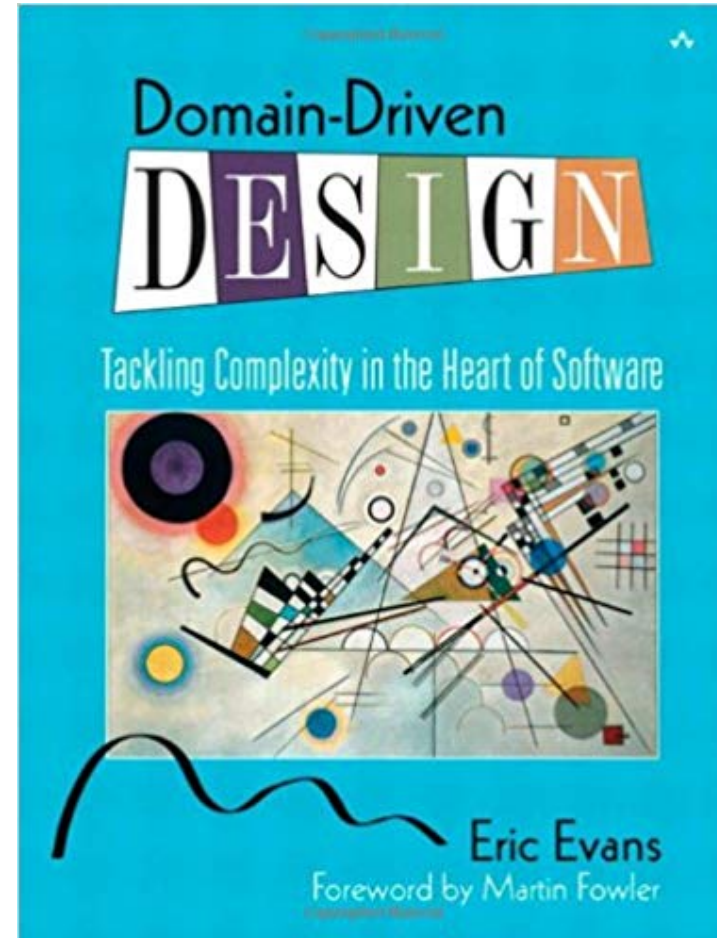
- Gebruik jouw favoriete taal: C#, Java, Python, ...
- Copilot / Claude Code
- Stel vragen wanneer je wil.
- Pauze

Dag 1 — De bouwstenen in code

DDD: Tackling Complexity

"A good domain model can be incredibly valuable, but it's not something that's easy to make. Few people can do it well, and it's very hard to teach."

— Martin Fowler, in foreword of "Domain-Driven Design" (Eric Evans)



Tactical vs Strategic

Splitsing tussen **strategic** design (grenzen, taal, proces) en **tactical** design (code-bouwstenen).

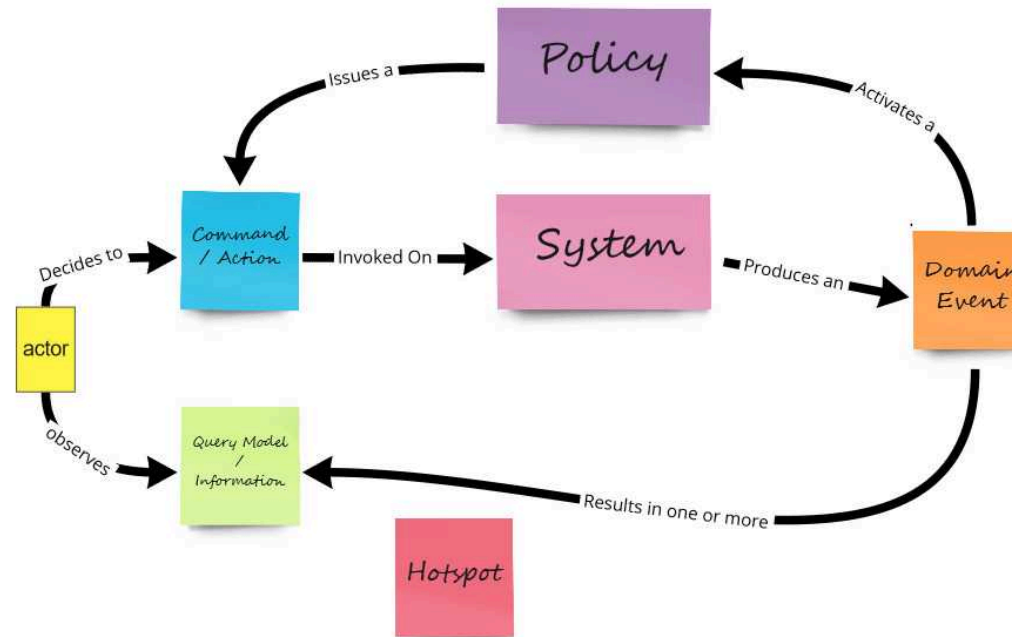
Strategic DDD (bounded contexts, event storming) kwam in de vorige sessie aan bod; vandaag vertalen we die output naar de tactical bouwstenen.

Prerequisites: deze workshop bouwt voort op de sessie **strategic DDD** (bounded contexts, event storming). De strategic output van toen is de input voor de tactical bouwstenen van vandaag.



Strategic

Process modelling Picture that explains "Almost" Everything



Example



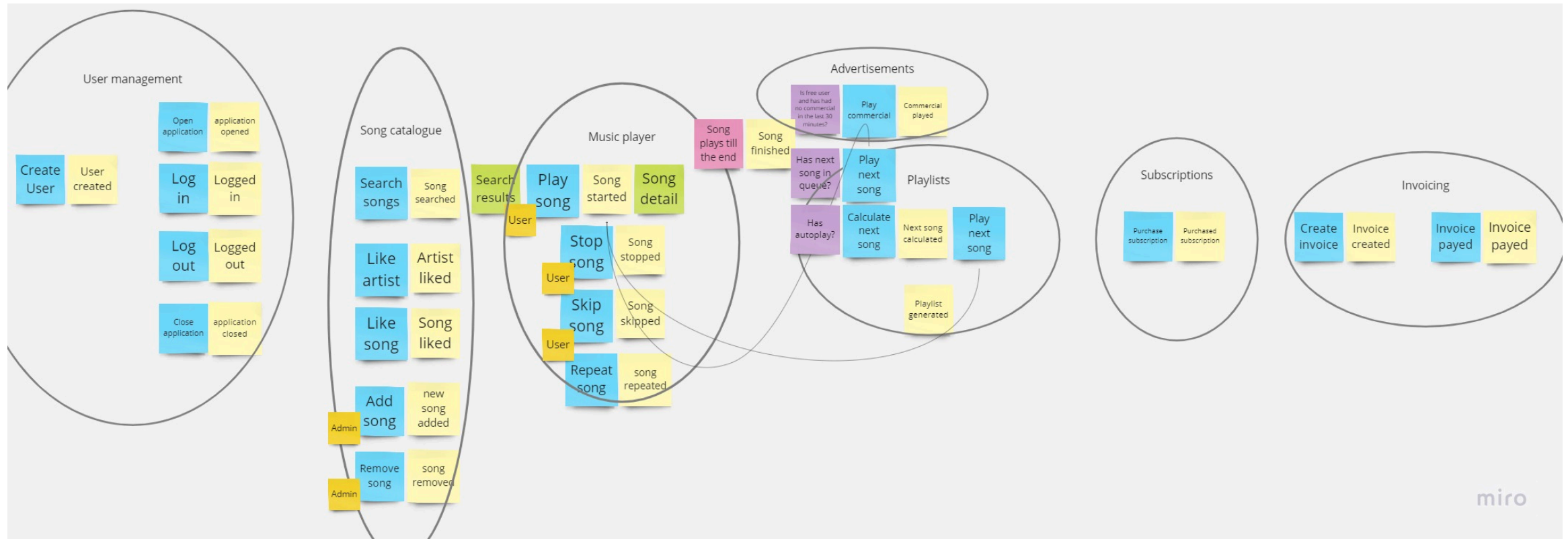
Event storming

- Wordt uitgevoerd met mensen uit de hele business
- Creëert een representatie van een businessproces
- Representeert waar de bottlenecks zitten
- Legt je ubiquitous language vast

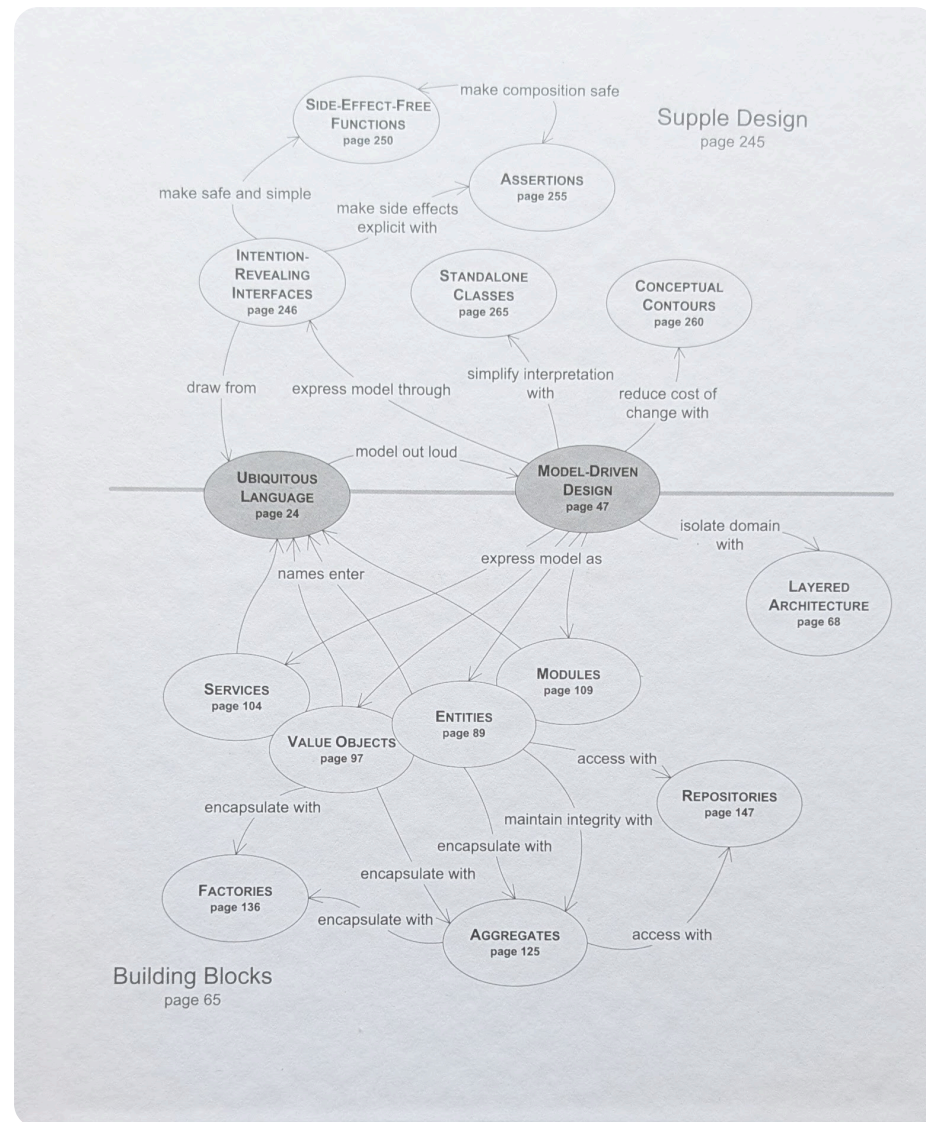
Event storming — voorbeeldbord

Ter oriëntatie op een event-stormingbord (gangbare conventies):

■ blauw = commands ■ geel = domain events ■ oranje = gebruikers ■ roze = external systems ○ cirkel = aggregates



Op de foto domineren inderdaad gele en blauwe stickies — de domain events en commands.

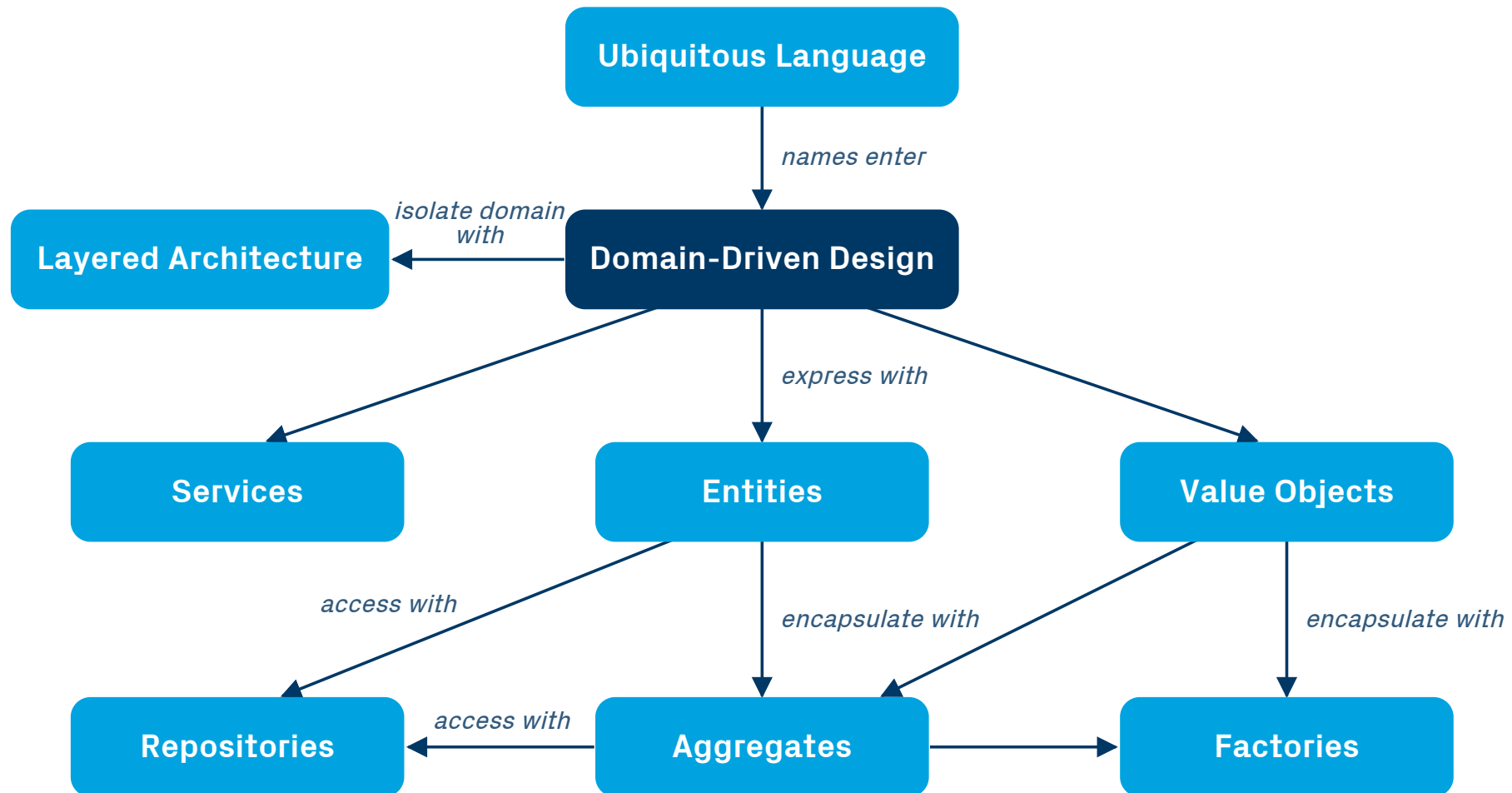


Tactical

Building blocks

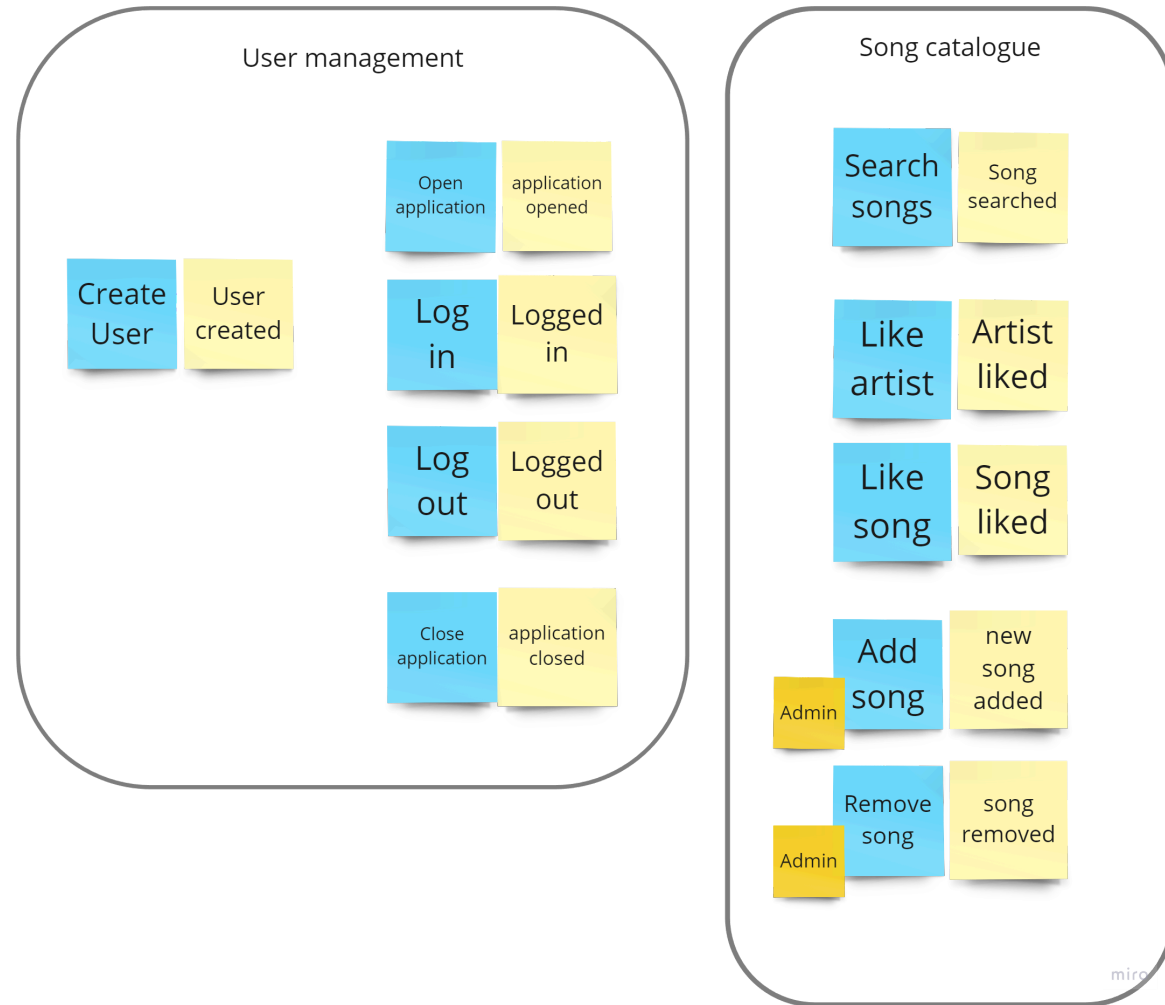
DDD building blocks

De kaart die door de hele dag als ankerpunt dient.



Ons domain

Intro tot het domein van de workshopapplicatie
(**HarmonyTunes**: gebruikers, albums, liedjes).



■ commands ■ domain events ○ aggregates ■ gebruikers

Ubiquitous language

- Eén gedeelde taal voor business én code: dezelfde woorden in gesprekken, modellen en code.
- Vandaag maak je je eigen ubiquitous language: kies namen voor je aggregates, commands, events en value objects.
- **Wees consistent:** één term per concept, overal gebruikt — geen synoniemen (User / Customer / Member).

DAG 1 · LAB 1

Oefening 1

- Maak een console applicatie om het programma te runnen later.
- Maak een model voor de users van onze applicatie.
- Maak een model voor songs in onze applicatie.
- Voeg de nodige properties toe aan de modellen.

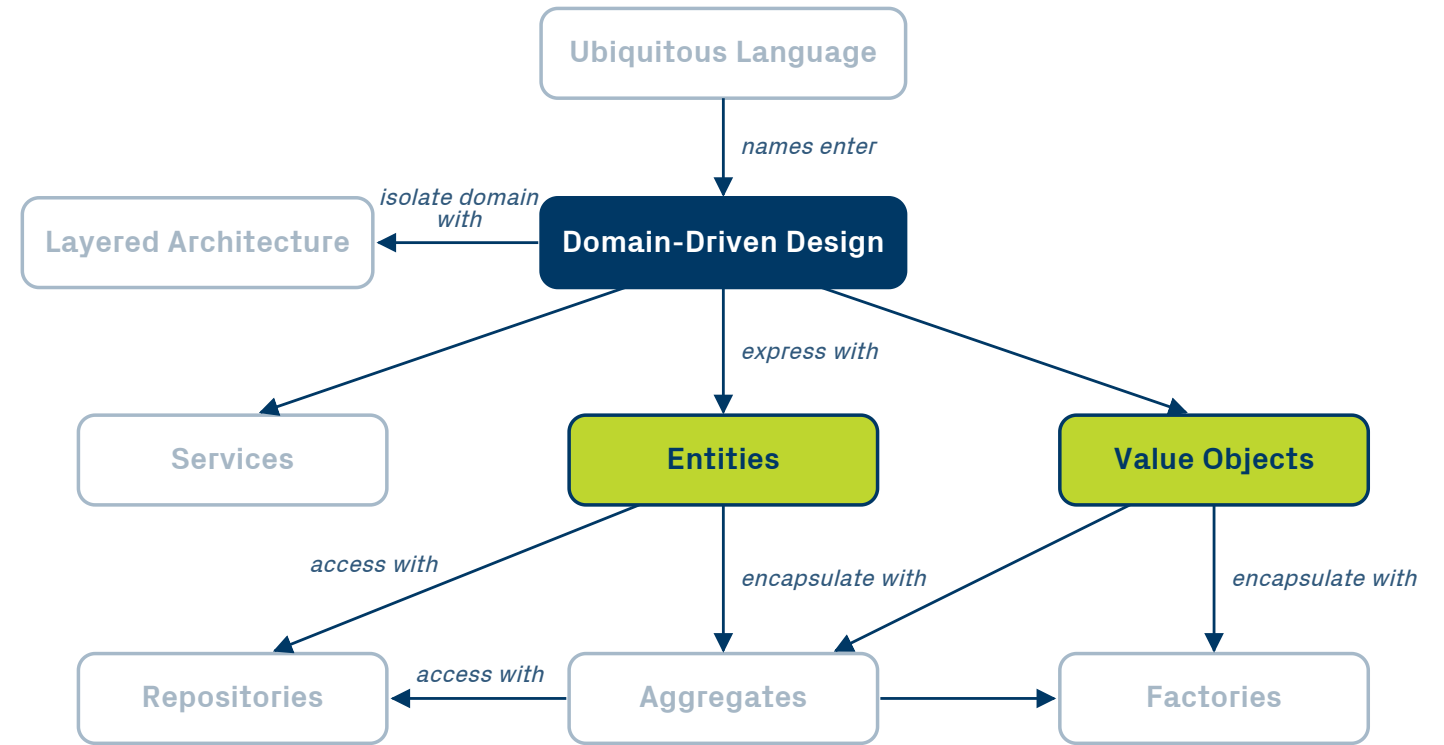
15 min

STARTER

Dag 1/DDD.Domain

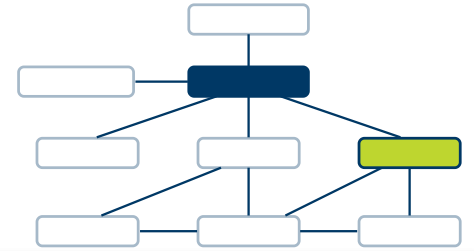
Klaar wanneer: je User- en Song-modellen compileren, hun properties dragen en je ze kan aanmaken.

Value objects en entities



Value objects

- "Represent a descriptive aspect of the domain with no conceptual identity."
- Immutable (onveranderlijk)
- Wordt getypeerd door zijn properties
- Simpel design



```
public class Age
{
    public int Value { get; }

    public Age(int age)
    {
        Value = age;
    }

    public override bool Equals(object obj)
    {
        if (obj == null || GetType() != obj.GetType())
        {
            return false;
        }

        return Value == ((Age)obj).Value;
    }
}
```

Value objects — met validatie

```
1 public class Age
2 {
3     public int Value { get; }
4
5     public Age(int age)
6     {
7         if (age is < 0 or > 130)
8         {
9             throw new InvalidAge(age);
10        }
11
12        Value = age;
13    }
14
15    public override bool Equals(object obj)
16    {
17        if (obj == null || GetType() != obj.GetType())
```

Value objects — met validatie

```
1 public class Age
2 {
3     public int Value { get; }
4
5     public Age(int age)
6     {
7         if (age is < 0 or > 130)
8         {
9             throw new InvalidAge(age);
10        }
11
12        Value = age;
13    }
14
15    public override bool Equals(object obj)
16    {
17        if (obj == null || GetType() != obj.GetType())
```

Value objects — met validatie

```
11
12     Value = age;
13 }
14
15 public override bool Equals(object obj)
16 {
17     if (obj == null || GetType() != obj.GetType())
18     {
19         return false;
20     }
21
22     return Value == ((Age)obj).Value;
23 }
24
25 public override int GetHashCode() => Value.GetHashCode();
26
27 public static bool operator ==(Age left, Age right) => Equals(left, right);
```

Value objects — met validatie

```
13     }
14
15     public override bool Equals(object obj)
16     {
17         if (obj == null || GetType() != obj.GetType())
18         {
19             return false;
20         }
21
22         return Value == ((Age)obj).Value;
23     }
24
25     public override int GetHashCode() => Value.GetHashCode();
26
27     public static bool operator ==(Age left, Age right) => Equals(left, right);
28     public static bool operator !=(Age left, Age right) => !Equals(left, right);
29 }
```

Value objects — Opgelet

Opgelet: `Equals` overschrijven zonder `GetHashCode` breekt `HashSet`/`Dictionary`-gedrag — twee "gelijke" objects landen dan in één set. `==` / `!=` vergelijken standaard nog op referentie.

Value objects als record

In modern C# is "immutable, getypeerd door properties, compare by value" precies wat een `record` doet — `Equals`, `GetHashCode`, `==` en `ToString` krijg je gratis.

Vergelijk met `Year / AlbumName` in de HarmonyTunes-code: zelfde idee, zelfde regels, zonder de `Equals / GetHashCode`-boilerplate.

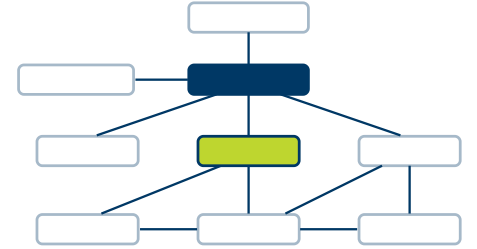
```
public sealed record Year
{
    public int Value { get; }

    public Year(int value)
    {
        if (value is < 1900 or > 2100)
        {
            throw new InvalidYear(value);
        }

        Value = value;
    }
}
```

Entity

- "Objects which have continuity through a life cycle and are being distinct independent of its attribute values over time."
- Mutable (aanpasbaar)
- Uniek identificeerbaar binnen het domain



```
public class User
{
    public UserId Id { get; }
    public Name Name { get; }
    public Age Age { get; }
    public Address Address { get; }

    public User(UserId id, Name name, Age age, Address address)
    {
        Id = id;
        Name = name;
        Age = age;
        Address = address;
    }
}
```

Identity

Globaal uniek

Toegewezen of gegenereerd tijdens creatie.

Lokaal uniek

Bv. liedje van een album; lokaal toegewezen of gegenereerd.

DAG 1 · LAB 2

Oefening 2

- Pas de modellen van oefening 1 aan en gebruik value objects en entities.
- Voeg de nodige validatie toe op de value objects.
- Maak entities aan. Wat merk je?

Vergelijk met je burens: **1-2-4-All** | 10 min

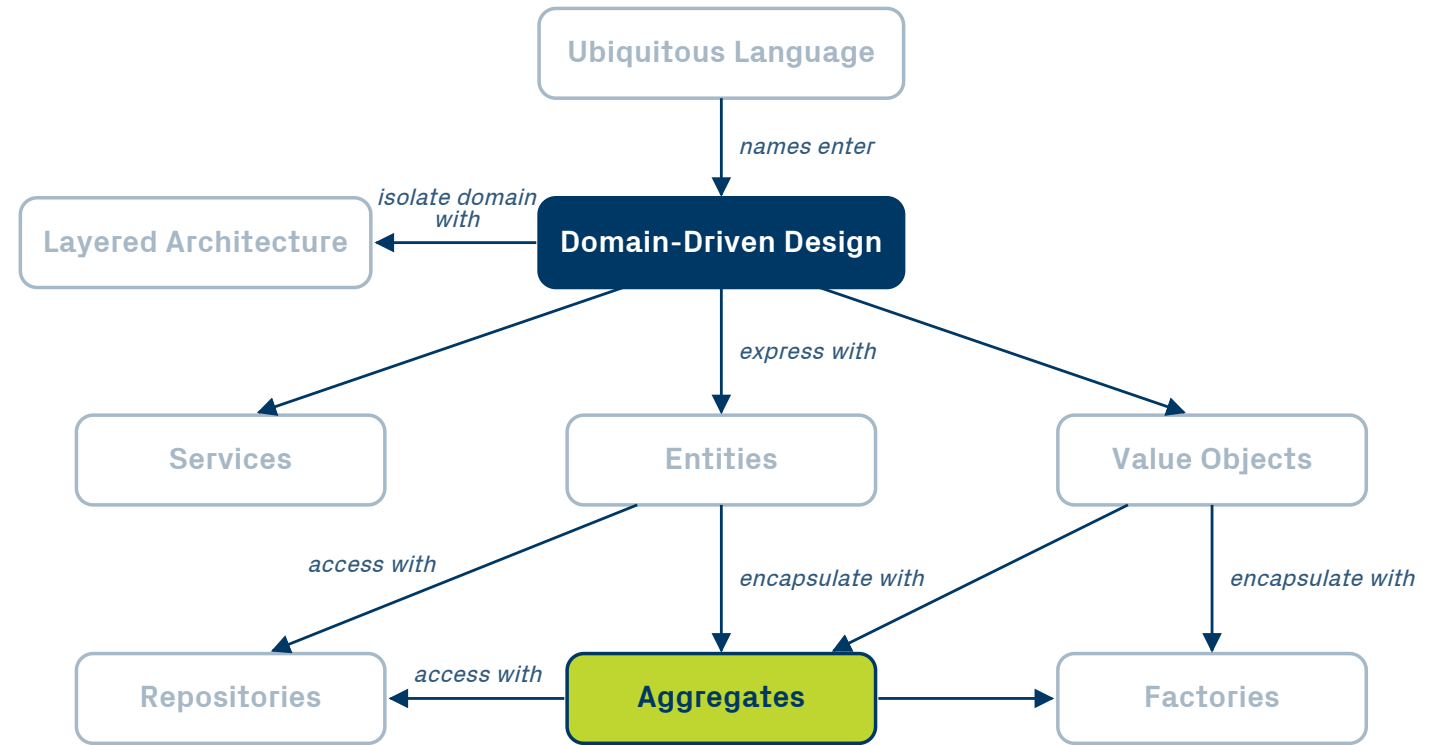
30 min

STARTER

Dag 1/DDD.Domain — verder dan lab 1

Klaar wanneer: ongeldige waarden in een value object worden geweigerd in de constructor, en twee inhoudelijk gelijke value objects als gelijk worden beschouwd (compare by value).

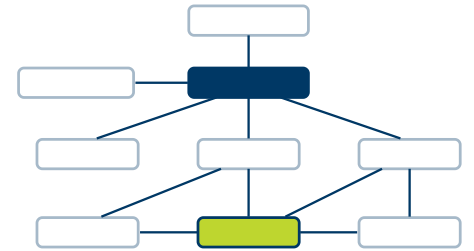
Aggregates



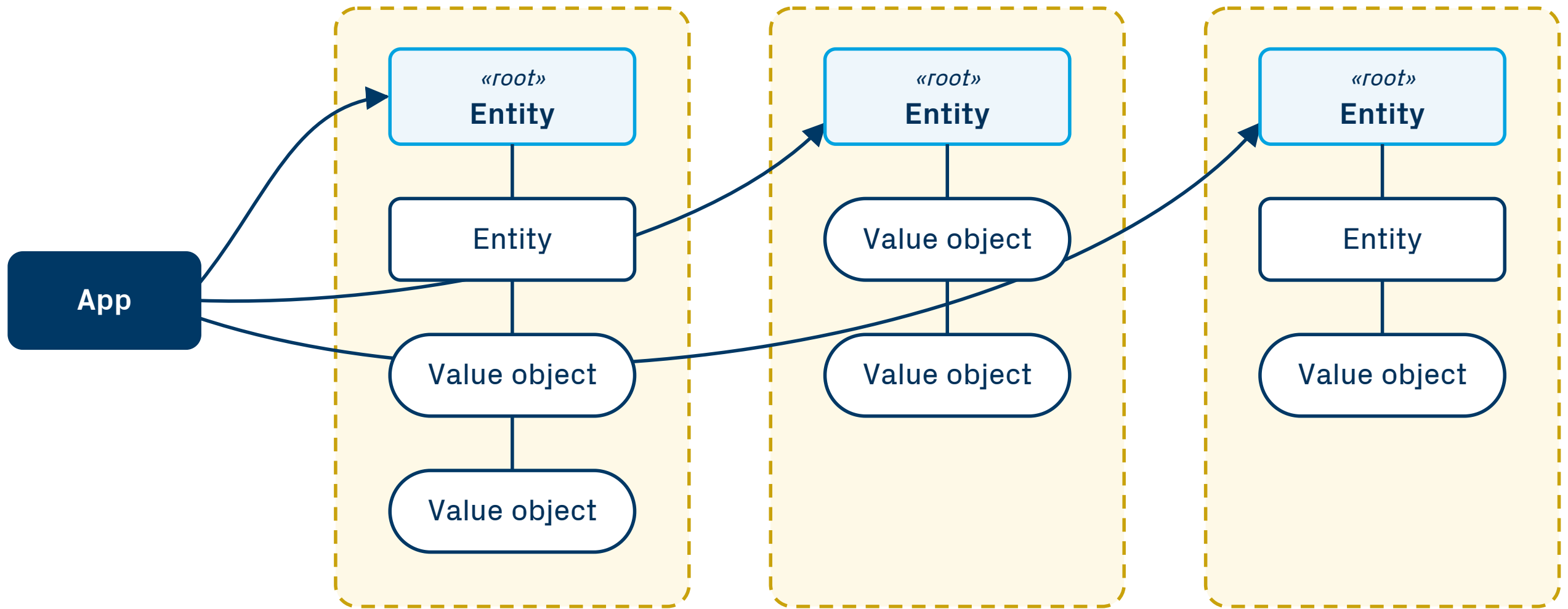
Aggregates

Definitie:

- "A cluster of associated objects that is treated as a unit for the purpose of data changes."
- Zijn verantwoordelijk voor hun eigen consistency.
- Een verzameling van entities en value objects.
- Er is exact **1 Aggregate Root** (entity): verantwoordelijk voor de consistency; enige toegang voor changes.



Aggregates — voorbeeld



Drie aggregate-kaders, elk met één «root»-entity — een externe App heeft toegang tot enkel de roots.

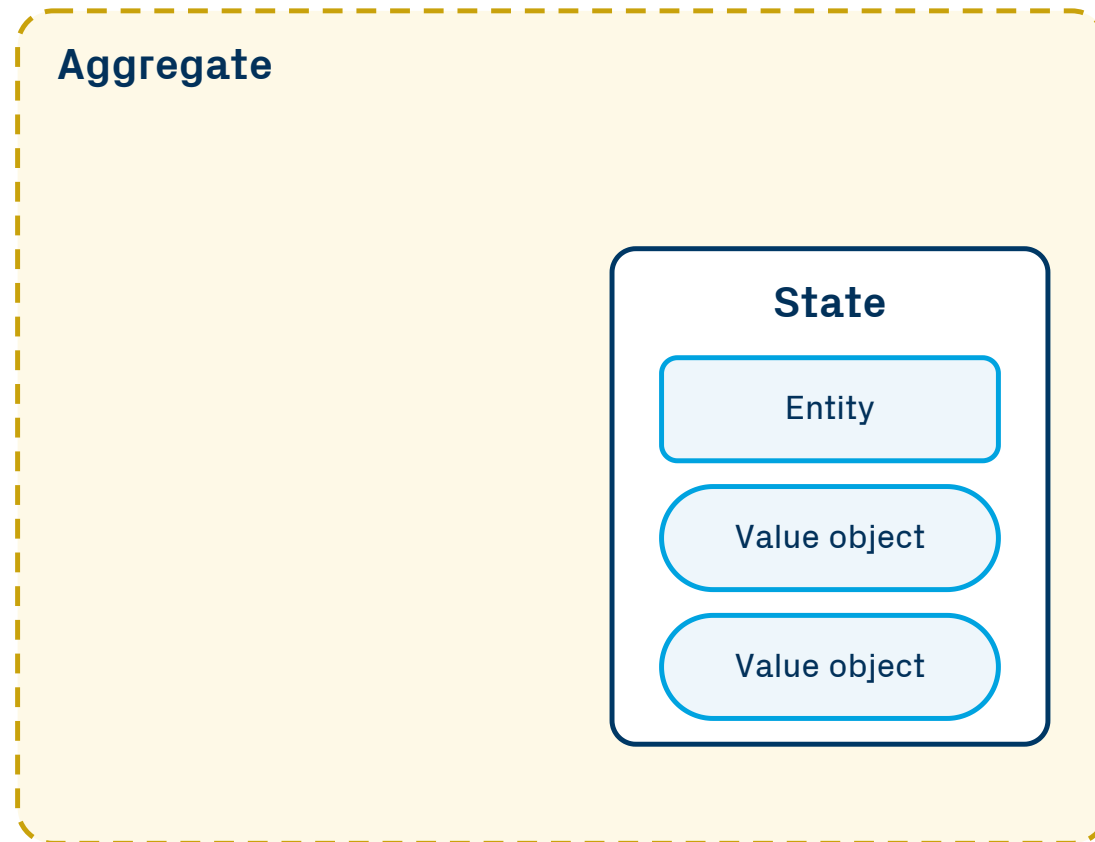
Aggregates — opbouw



Aggregate

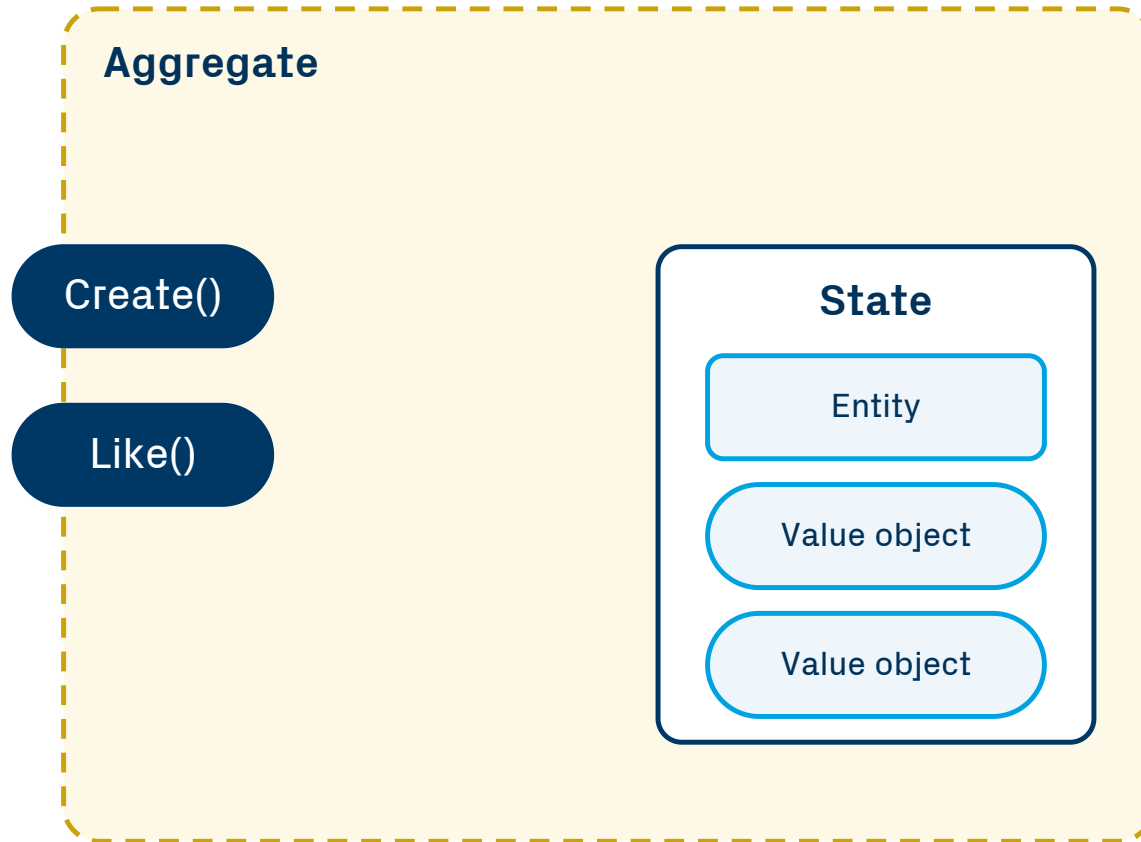
1 Aggregate (kader)

Aggregates — opbouw



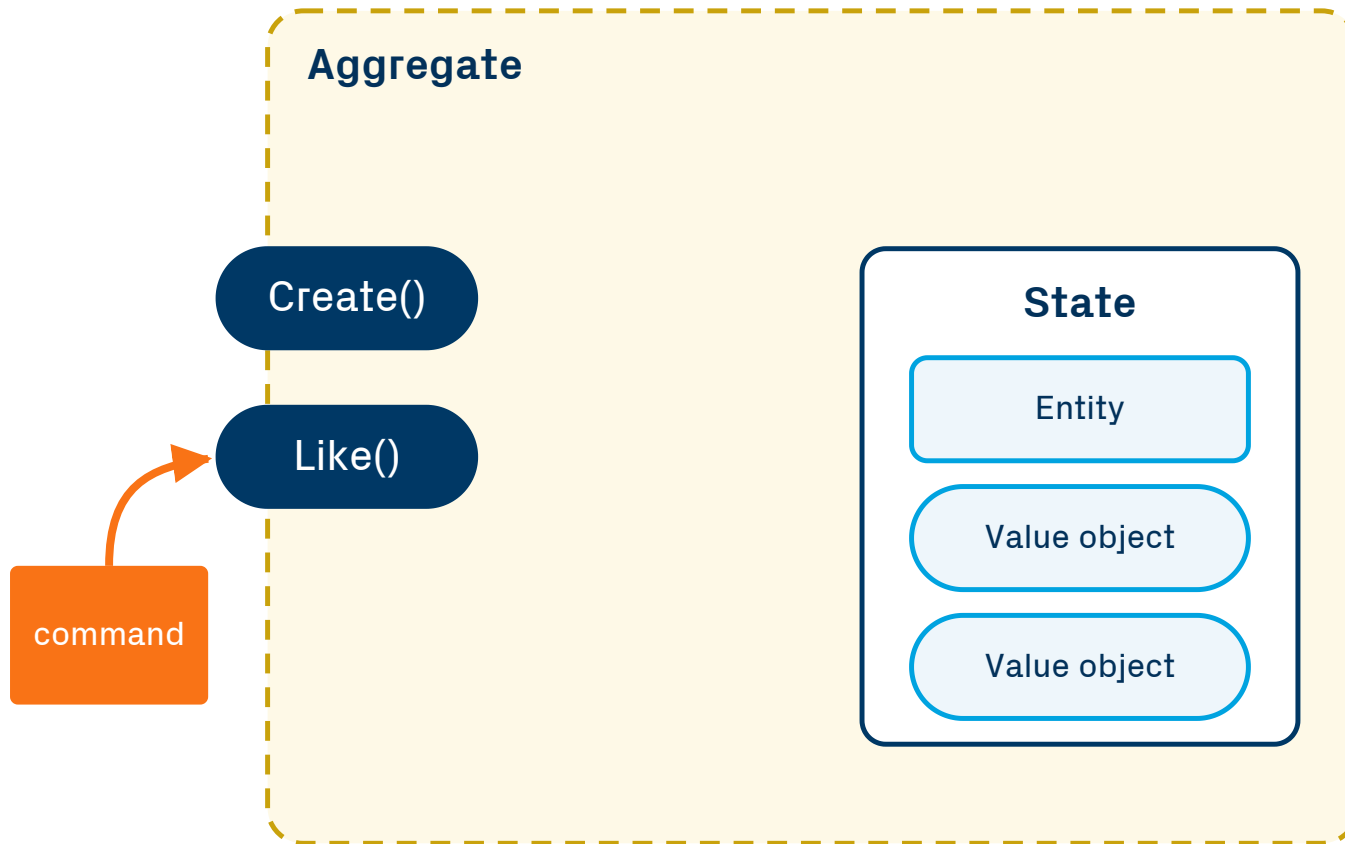
- 1 **Aggregate** (kader)
- 2 + **State, entities en value objects**

Aggregates — opbouw



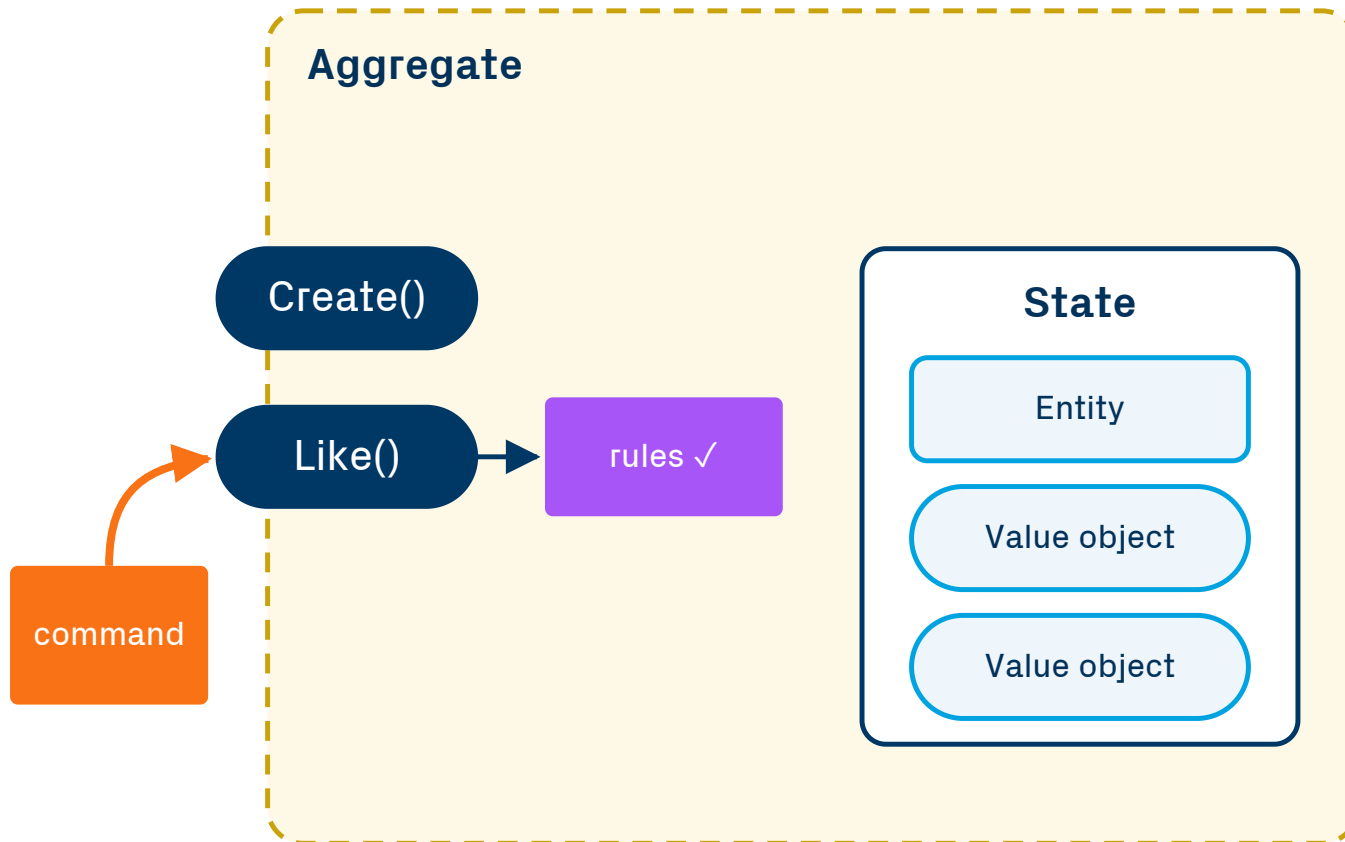
- 1 **Aggregate** (kader)
- 2 + **State**, entities en value objects
- 3 + **Public methods**

Aggregates — opbouw



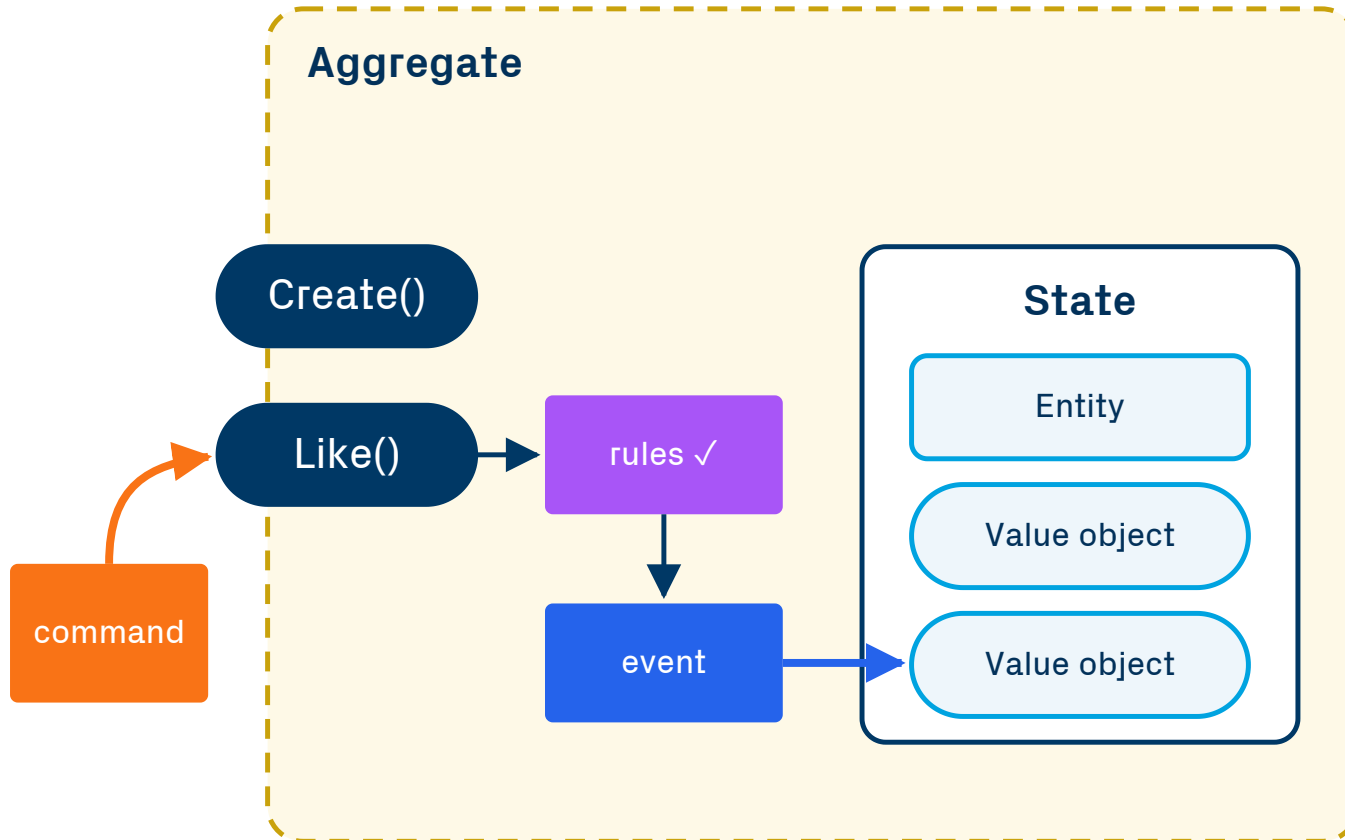
- 1 **Aggregate** (kader)
- 2 + **State**, entities en value objects
- 3 + **Public methods**
- 4 + **Domain command** (komt binnen via public methods)

Aggregates — opbouw



- 1 **Aggregate** (kader)
- 2 + **State**, entities en value objects
- 3 + **Public methods**
- 4 + **Domain command** (komt binnen via public methods)
- 5 + **Domain rules** (validatie bij command)

Aggregates — opbouw



- 1 **Aggregate** (kader)
- 2 + **State**, entities en value objects
- 3 + **Public methods**
- 4 + **Domain command** (komt binnen via public methods)
- 5 + **Domain rules** (validatie bij command)
- 6 + **Domain events** (muteren van de state)

Aggregate-ontwerpregels

Naast *wat* een aggregate is, ook *hoe groot* en *hoe gescheiden*:

Eén aggregate = **één consistentiegrens**; houd aggregates klein.

Verwijs naar andere aggregates **per id**, nooit via objectreferenties.

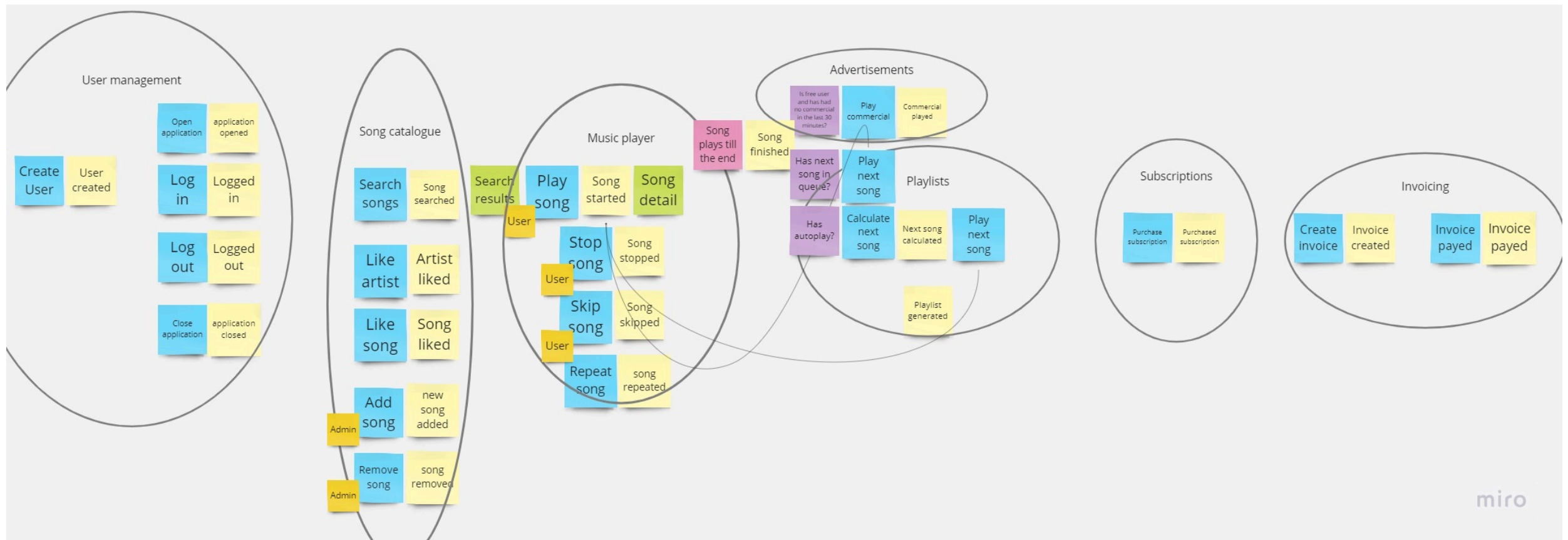
Eén transactie wijzigt **één** aggregate; effecten op andere aggregates gebeuren via events (eventual consistency).

Een repository bestaat **per aggregate root** — dus geen `AlbumSongRepository`.

Aggregates — herken de blokken

Hetzelfde event-stormingbord als hiervoor: herken de blokken die je in de volgende oefening gaat bouwen.

■ commands ■ events ○ aggregates ■ policies/regels



Hands on exploration

- Creatie van aggregates
- Afhandelen van commando's (domain commands)
- Validatie met domain rules
- Muteren van de state met domain events

DAG 1 · LAB 3

Oefening 3

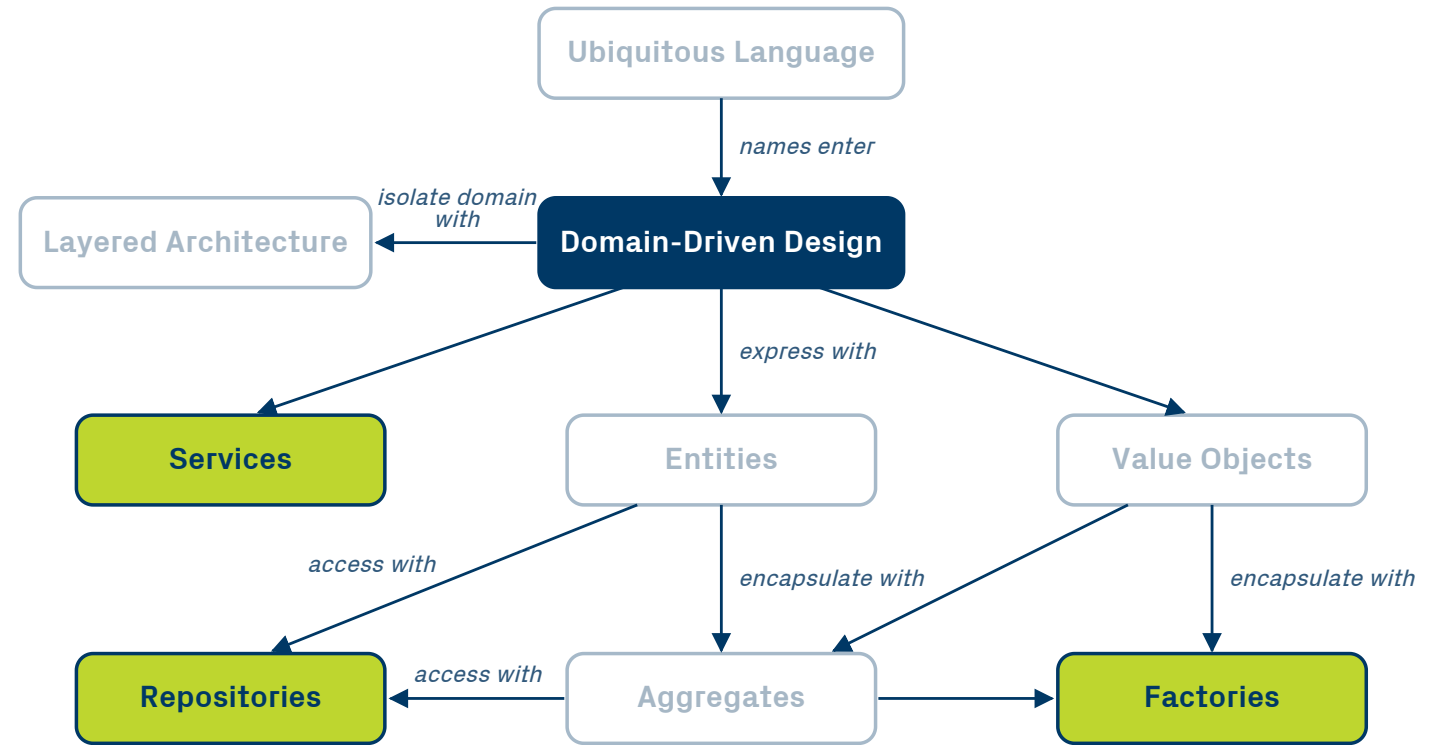
- Creëer aggregates en begin met User of Song.
- Implementeer per aggregate commands en events.

STARTER

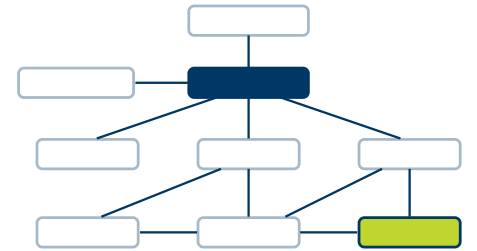
`Dag 1/DDD.Domain` — verder dan lab 2

Klaar wanneer: een command de state mutert via events, en een overtreden domain rule een exception gooit.

Factories, repositories en services



Factories

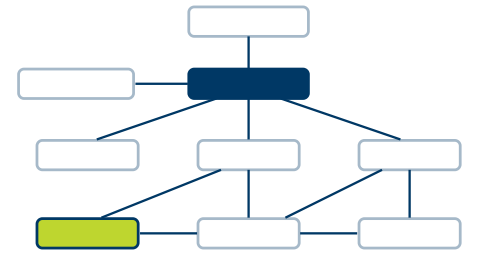


- "Aggregates can become complex object graphs of which the construction should be the responsibility of factories."
- Factories worden gebruikt wanneer normale creatie van een value object of entity te complex wordt.
- Gebruiken vaak DI om services aan te roepen tijdens creatie.

Repositories

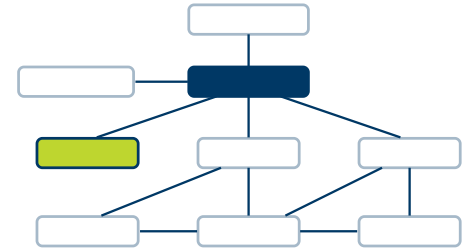
- "Represent all objects of a certain type as a collection with querying capabilities."
- Houdt entities bij.
- Kan connecteren met: database, cache, text file, ...

Onthoud de ontwerpregel: **per aggregate root**, niet per entiteit.



Services

- "A process or transformation in the domain which is not a natural responsibility of an Entity or a Value Object."
- Domain services helpen met: het encapsuleren van domain logica; het ophalen van informatie binnen of buiten het domein dat nodig is voor validatie; ...



Voorbeeld — domain service via DI in een command:

```
public void ChangeName(Name name, INameValidator nameValidator,
                        INameQueryService nameQueryService)
{
    if (!nameValidator.IsValid(name))
    {
        throw new InvalidNameException("Invalid name");
    }
    if (nameQueryService.Exists(name))
    {
        throw new NameAlreadyExistsException("Name already exists");
    }
    ...
}
```

DAG 1 · LAB 4

Oefening 4

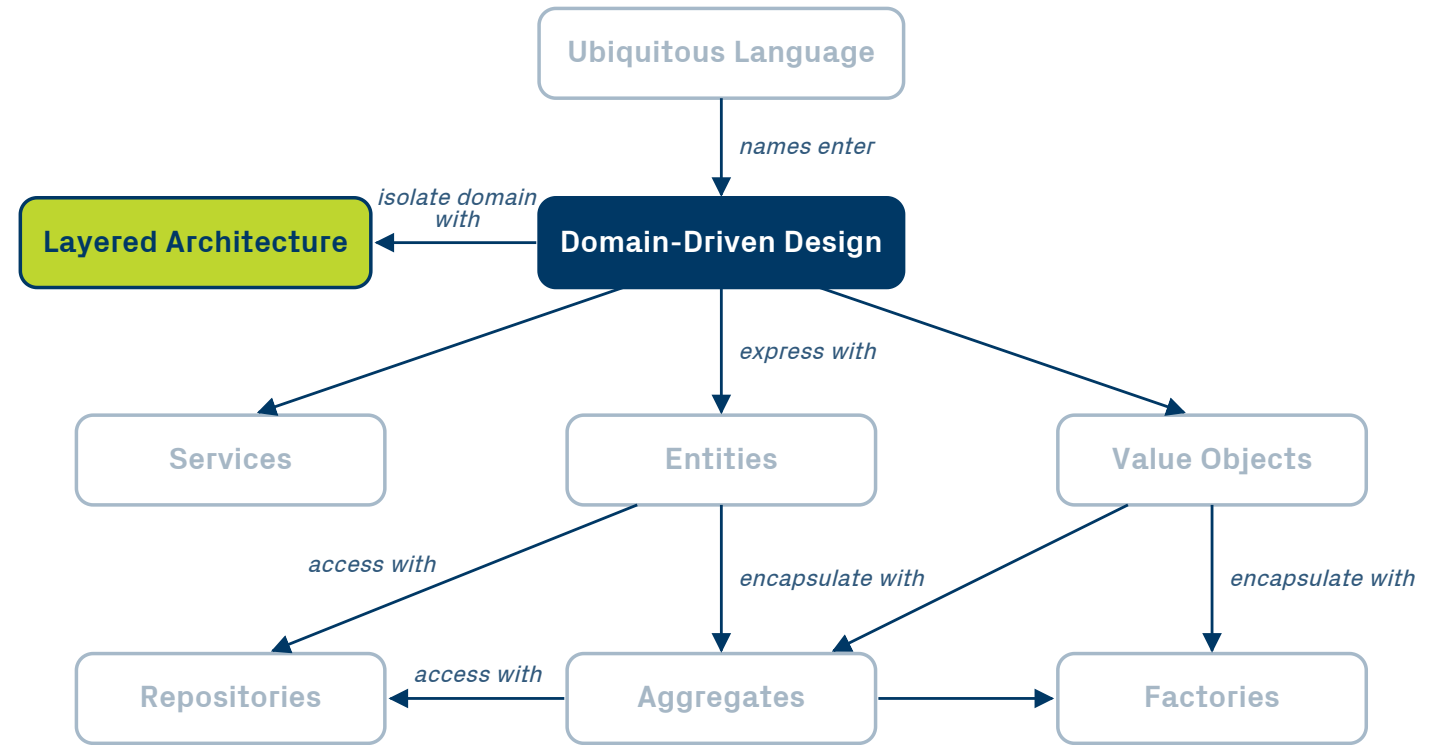
- Voeg domain services toe aan je aggregates of value objects.
- Voeg een factory toe voor een value object.
- Wat merk je op?

STARTER

Dag 1/DDD.Domain — verder dan lab 3

Klaar wanneer: een domain service een regel afhandelt die niet in de aggregate thuishoort, en een value object via een factory gecreëerd wordt.

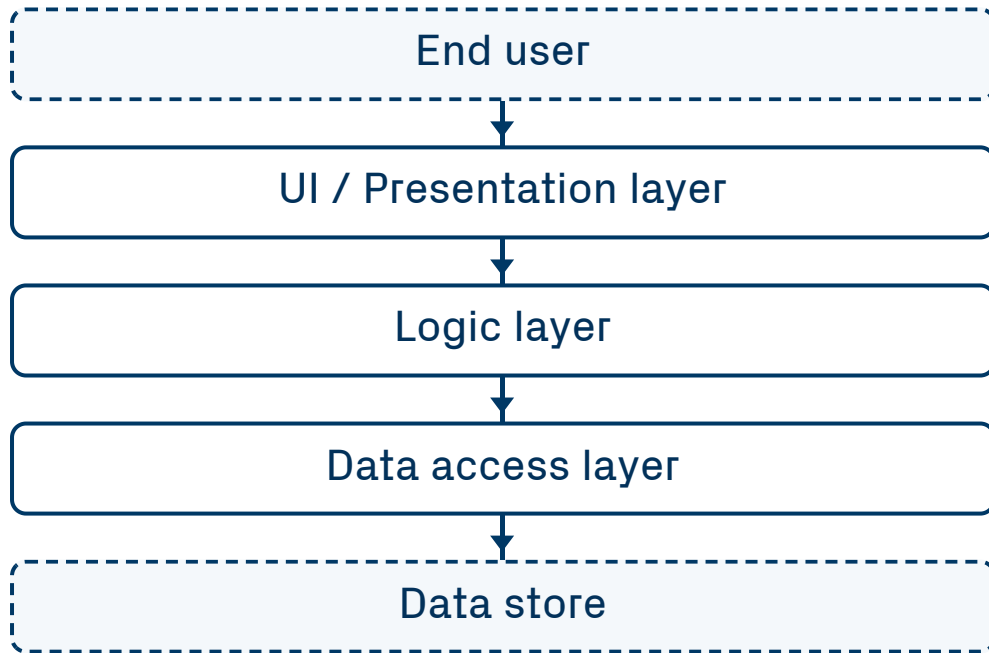
Layered Architecture



Layered Architecture

Onion / hexagonal — Domain Core in het centrum

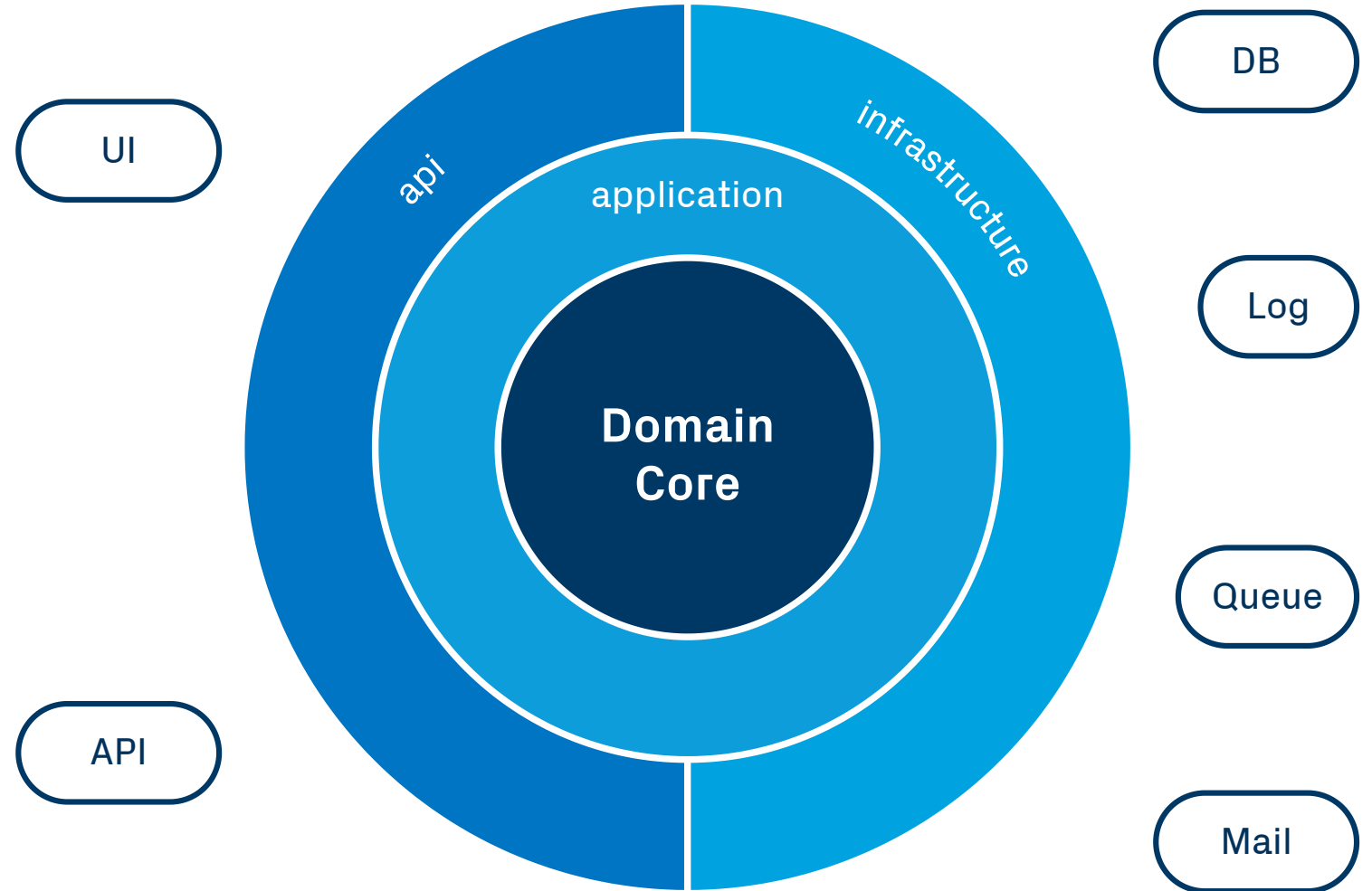
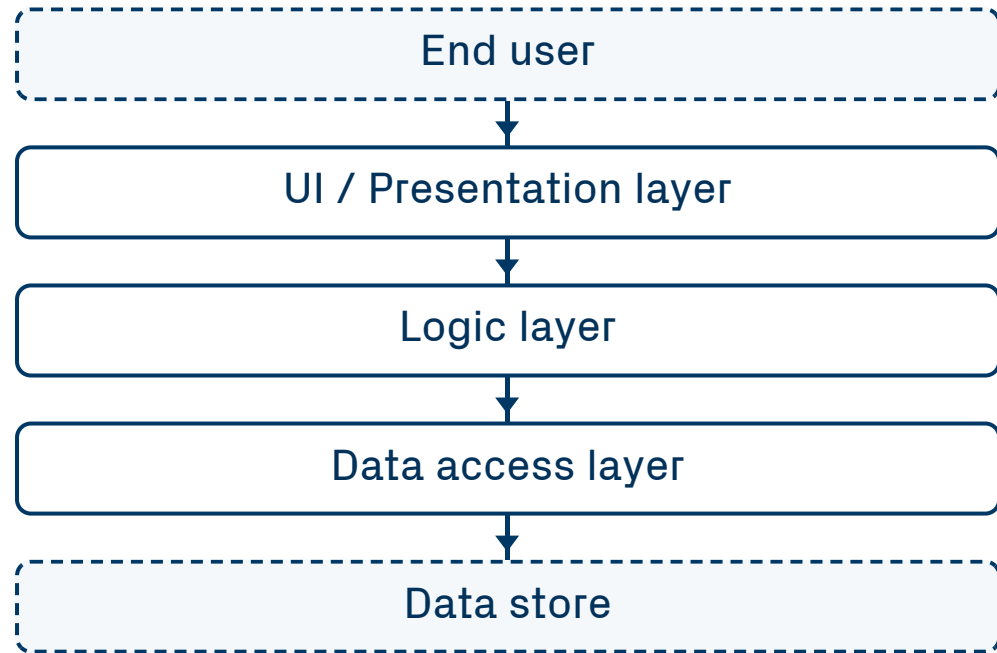
Klassieke lagen



Layered Architecture

Onion / hexagonal — Domain Core in het centrum

Klassieke lagen



Layered Architecture — principles

- Onion/hexagonal architecture zorgt voor betere separation of concerns
 - Domain logica staat centraal
 - Afhankelijkheid is naar binnen toe (inversion of control)
 - Domain logica is niet afhankelijk van de infrastructuur
- Alles is losgekoppeld via interfaces
 - Interfaces beschreven in de application- en domain layer
 - Losse koppeling met de concrete implementatie

Application services

- Use case oriented — gefocust op het vervullen van een business use case
- Bevat geen domain logica
- Allerlaatste moment waarop er geconverteerd wordt naar domain types
- Beschrijft een proces; voornamelijk stateless services

Typisch voltooit een application service de volgende stappen:

Application services

- Use case oriented — gefocust op het vervullen van een business use case
- Bevat geen domain logica
- Allerlaatste moment waarop er geconverteerd wordt naar domain types
- Beschrijft een proces; voornamelijk stateless services

Typisch voltooit een application service de volgende stappen:

- 1 Validatie van de input

Application services

- Use case oriented — gefocust op het vervullen van een business use case
- Bevat geen domain logica
- Allerlaatste moment waarop er geconverteerd wordt naar domain types
- Beschrijft een proces; voornamelijk stateless services

Typisch voltooit een application service de volgende stappen:

- 1 Validatie van de input
- 2 Ophalen van domain entities

Application services

- Use case oriented — gefocust op het vervullen van een business use case
- Bevat geen domain logica
- Allerlaatste moment waarop er geconverteerd wordt naar domain types
- Beschrijft een proces; voornamelijk stateless services

Typisch voltooit een application service de volgende stappen:

- 1 Validatie van de input
- 2 Ophalen van domain entities
- 3 Uitvoeren van domain commands op een aggregate

Application services

- Use case oriented — gefocust op het vervullen van een business use case
- Bevat geen domain logica
- Allerlaatste moment waarop er geconverteerd wordt naar domain types
- Beschrijft een proces; voornamelijk stateless services

Typisch voltooit een application service de volgende stappen:

- 1 Validatie van de input
- 2 Ophalen van domain entities
- 3 Uitvoeren van domain commands op een aggregate
- 4 Opslaan van domain entities

Application services

- Use case oriented — gefocust op het vervullen van een business use case
- Bevat geen domain logica
- Allerlaatste moment waarop er geconverteerd wordt naar domain types
- Beschrijft een proces; voornamelijk stateless services

Typisch voltooit een application service de volgende stappen:

- 1 Validatie van de input
- 2 Ophalen van domain entities
- 3 Uitvoeren van domain commands op een aggregate
- 4 Opslaan van domain entities
- 5 Notificaties verzenden van belangrijke events

Testen van aggregates

Het events-based ontwerp heeft een testersvoordeel: een command testen = **asserter de emitted events**. Geen mock van de state nodig — `AggregateRoot.Events` geeft de events na een command.

```
[Fact]
public void Like_EmitsAlbumLikedEvent()
{
    var album = new AlbumAggregate(new AlbumReference(Guid.NewGuid()));
    album.Create(new AlbumName("Meteora"));

    album.Like();

    Assert.Contains(album.Events, e => e is AlbumLiked);
}
```

Testen — regel geschonden

```
[Fact]
public void Create_ThrowsAlbumAlreadyCreated_WhenCalledOnExistingAl
{
    var album = new AlbumAggregate(new AlbumReference(Guid.NewGuid(
album.Create(new AlbumName("Meteora")));

    Assert.Throws<AlbumAlreadyCreated>(
        () => album.Create(new AlbumName("Use Your Illusion")));
}
```

Domeinlogica heeft geen infrastructuur nodig
— dus aggregatetesten hebben geen
database, HTTP of bus.

Mocken met NSubstitute

Waar mocks wél hun plaats hebben: de application service. Met **NSubstitute** substitutes je de repository en de message publisher:

```
[Fact]
public async Task CreateAlbum_publishes_AlbumUpdatedNotification()
{
    var repository = Substitute.For<IAlbumRepository>();
    var publisher = Substitute.For<IMessagePublisher>();
    var service = new AlbumService(repository, publisher);

    await service.CreateAlbum(new AlbumName("Meteora"));

    repository.Received(1).Add(Arg.Any<AlbumAggregate>());
    await publisher.Received(1)
        .Publish(Arg.Any<AlbumUpdatedNotification>());
}
```

Dit is precies het voordeel dat bij de hexagonal architectuur hoort: alles buiten het domein is een interface, dus overal substituuat in de test.

Application services — hands on exploration

- Maak een application-laag
- Maak een ports-laag

Projectstructuur

Verwachte opbouw van een C#-solution:

HarmonyTunes.slnx	
└─ HarmonyTunes.Domain.Core	gedeelte basisklassen
└─ Domain/	AggregateRoot, ValueObject, DomainEvent, ...
└─ Application/	IRepository, ApplicationService
└─ HarmonyTunes.Catalogue	domein + application, per aggregate
└─ Album/	
└─ Domain/	aggregate, state, Events/, ValueObjects/
└─ Application/	service, Interfaces/, Models/, Notifications/
└─ Artist/ · Song/	zelfde opbouw
└─ Shared/	references, IMessagePublisher
└─ HarmonyTunes.Ports	adapters (infrastructuur)
└─ Album/Adapters/	Database/, Messaging/, WebApi/
└─ Album/Models/	DTO's
└─ Program.cs	composition root

DAG 1 · LAB 5

Oefening 5

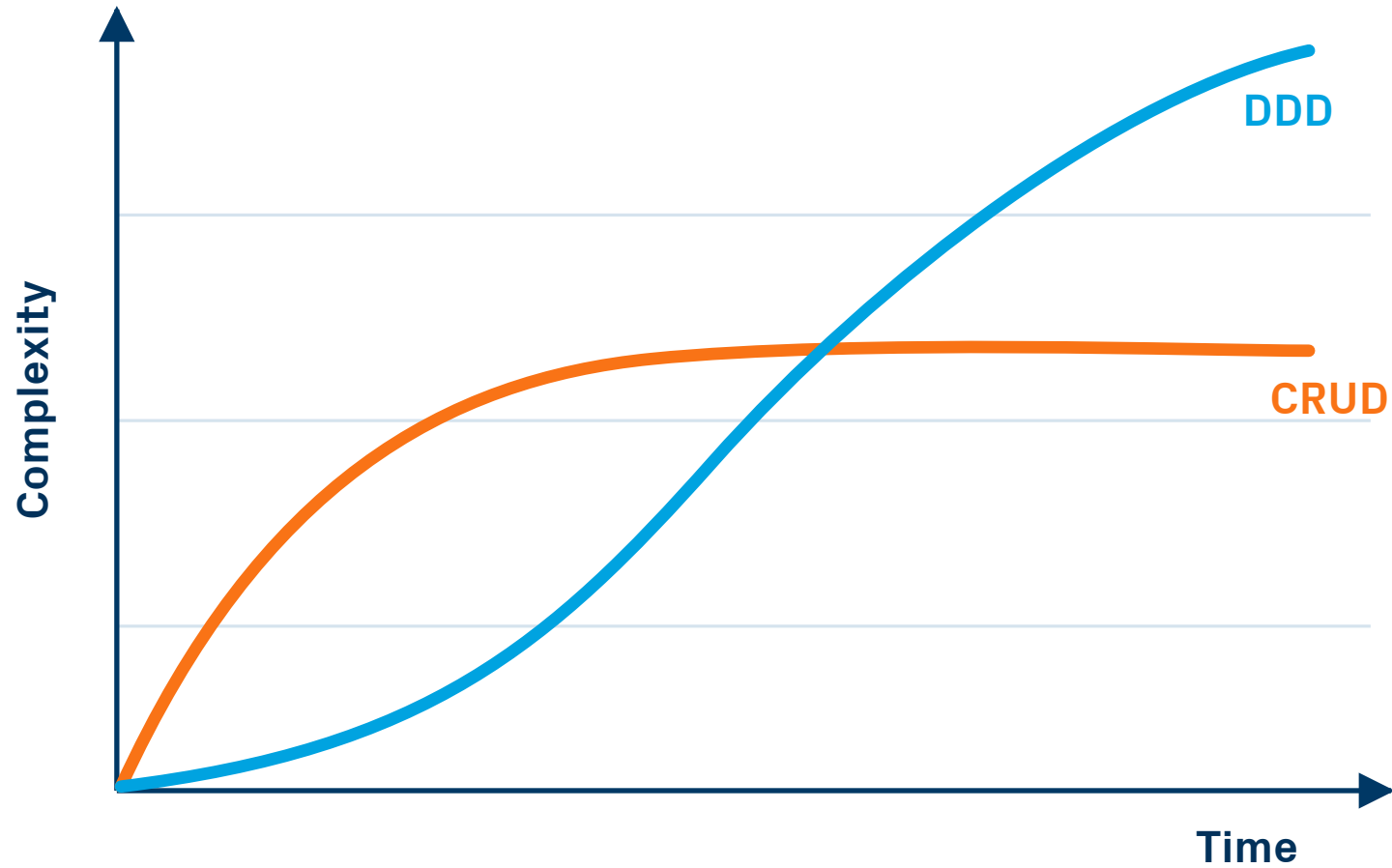
- Maak je eigen onion architecture rond je applicatie. Zorg voor duidelijke interfaces en juiste afhankelijkheden tussen lagen.
- **Verdieping in dag 2** (dag 2 → oefening 1): werk er daar verder aan door.

STARTER

Dag 1/DDD.Domain — verder dan lab 4

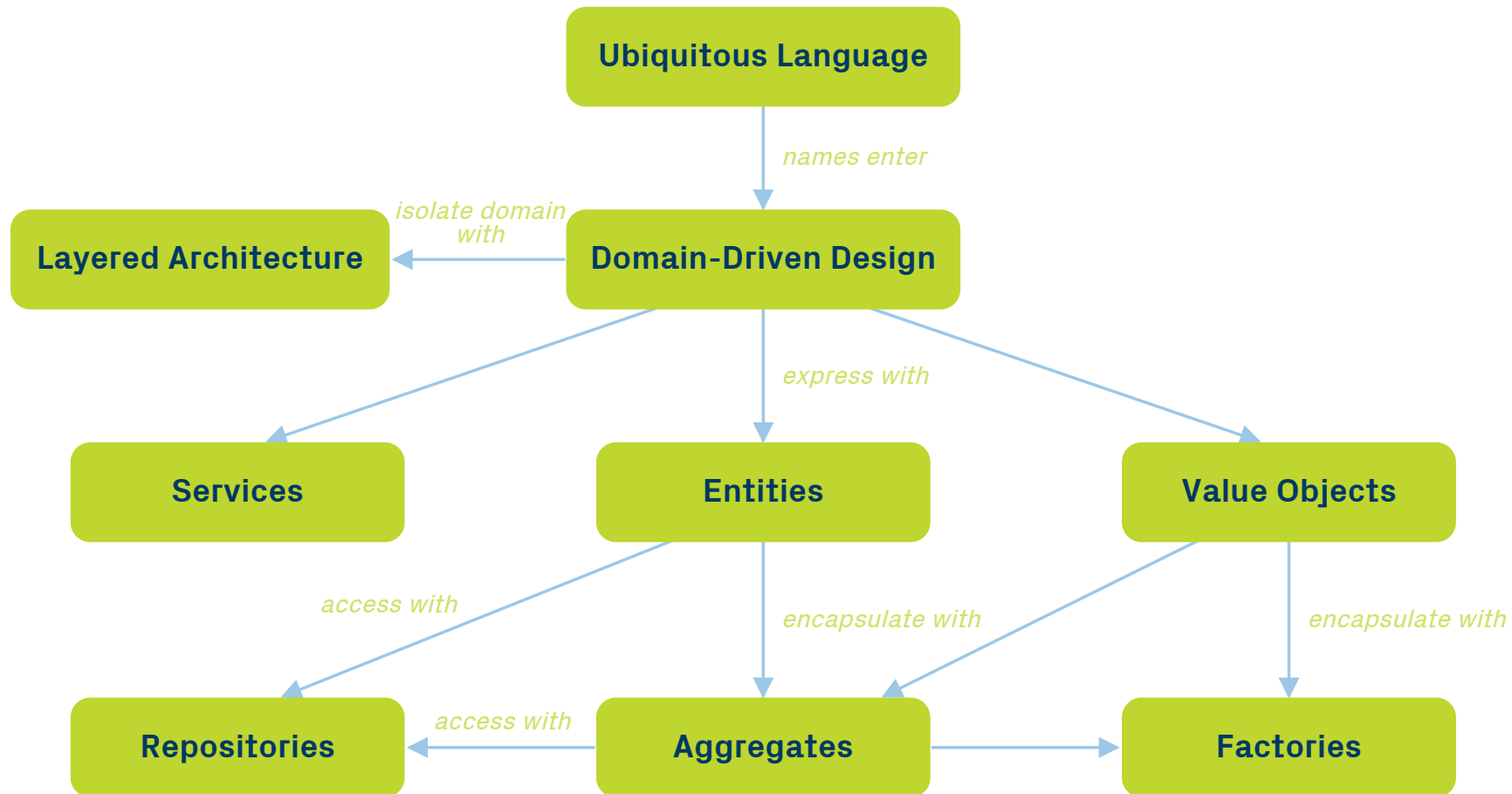
Klaar wanneer: het domein geen referenties heeft naar infrastructurele klassen, en application/presentation afhangen van domain via interfaces.

DDD versus CRUD



CRUD blijft vlak en loopt vast op complexiteit; DDD kost initieel meer tijd maar schaalt verder.

DDD building blocks — overzicht



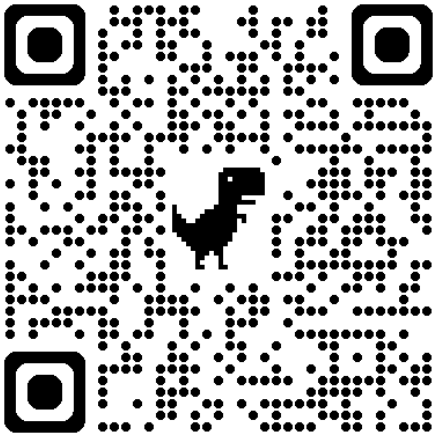
Samenvatting dag 1

Bouwsteen	Verantwoordelijkheid
Value object	Beschrijvend aspect zonder identiteit; immutable, compare by value
Entity	Continuïteit door een levenscyclus; unieke id binnen het domein
Aggregate	Eén consistentiegrens met één root; commands → rules → events
Factory	Complexe creatie van VOs/entities/aggregates
Repository	Aggregate roots bijhouden en opvragen, per root
Domain service	Domeinlogica die niet in entity/VO past
Application service	Use case orkestratie, geen domeinlogica
Onion/hexagonal	Domein centraal, afhankelijkheden naar binnen

Testen: command → asserteer `Events`; mocks enkel aan de interfaces van de application-laag.

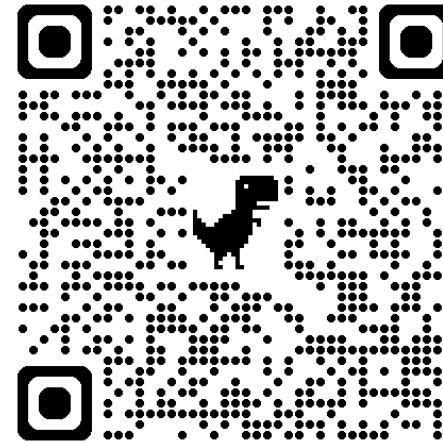
Info

Leuke programmeerwedstrijd op 24 oktober:
CodeBash 2026



QR-code — CodeBash

Nog op zoek naar een leuke, creatieve stage-
opdracht? Kom even praten tijdens de
handshake event.



QR-code — stage opdrachten

Afsluiting

Tot volgende week voor dag 2 —

Daniel.mertens@infosupport.com